

Automatic Playlist Continuation through a Composition of Collaborative Filters

Irene Teinemaa
University of Tartu
Tartu, Estonia
irene.teinemaa@ut.ee

Niek Tax
Eindhoven University of Technology
Eindhoven, Netherlands
n.tax@tue.nl

Carlos Bentes
STACC
Tartu, Estonia
carlos.bentes@stacc.ee

ABSTRACT

The RecSys Challenge 2018 focused on automatic playlist continuation, i.e., the task was to recommend additional music tracks for playlists based on the playlist's title and/or a subset of the tracks that it already contains. The challenge is based on the Spotify Million Playlist Dataset (MPD), containing the tracks and the metadata from one million real-life playlists. This paper describes the automatic playlist continuation solution of *team Latte*, which is based on a composition of collaborative filters that each capture different aspects of a playlist, where the optimal combination of those collaborative filters is determined using a Tree-structured Parzen Estimator (TPE). The solution obtained the 12th place out of 112 participating teams in the final leaderboard. Team Latte participated in the main track of the challenge of the RecSys Challenge 2018.

KEYWORDS

collaborative filter, hyperparameter optimization, music recommender, automatic playlist continuation

1 INTRODUCTION

With the increasing popularity of online music streaming services, the task of selecting relevant and personalized content in large music catalogs becomes important to avoid choice overload [5]. An open challenge in the area of personalization for music recommender systems is known as *automatic playlist continuation* (APC), where the task is to recommend tracks that are likely to be selected as additional tracks for an existing playlist. In APC it is important to recommend relevant content while, at the same time, respecting the characteristics of the original playlist [8]. For example, the recommended songs for the continuation of a playlist that consists of Christmas songs should be other Christmas songs.

To promote progress in the area of APC, the RecSys Challenge 2018 focuses on this task. The challenge was organized by Spotify, The University of Massachusetts, Amherst, and Johannes Kepler University, Linz, and was open for submissions from January to July 2018. In the competition, participants were challenged to create a recommendation system

for APC using a dataset of one million playlists that have been created by Spotify users in North America.

The competition task was to generate a list of 500 tracks as playlist continuation for each of the 10000 playlists in the challenge dataset. The playlists in the challenge set were divided into ten *challenge categories*, based on the number of seed tracks (the tracks that are already known to be present in the playlist) and the availability of the playlist title.

This manuscript describes the solution proposed by Team Latte. The main idea behind the proposed approach is to construct several collaborative filters, based on the co-occurrence of tracks with other tracks, artists, albums, and words in the playlist title. Furthermore, the solution adopts a specialized optimization strategy, where the weights of each collaborative filter are optimized locally within each challenge category using an optimization method called Tree-structured Parzen Estimator (TPE) [4]. The final recommendation scores are produced as a weighted sum of the individual collaborative filtering components, and then post-processed using several heuristic strategies.

The remainder of this paper is organized as follows: Section 2 describes the data provided during the competition. Section 3 describes the proposed framework based on several collaborative filters and their combination. Section 4 further details the model optimization and selection procedures to combine the collaborative filters, and discusses some results on internal validation sets. Finally, in Section 5 we conclude the paper.

2 DATASET

The dataset consists of one million playlists created by Spotify users and distributed as the Million Playlist Dataset (MPD) for exclusive use in the competition. The MPD dataset includes information about the playlist (title, identification, number of artists, playlist duration) and track information (album name, identification, artist, track duration, track name) for every playlist. A complete description of the dataset can be found in [2].

In addition to the MPD dataset, the organizers provided the *challenge dataset*, i.e. an official test dataset that contains partial information about 10000 playlists: the playlist title and/or a number of *seed tracks* (a subset of tracks present in

Table 1: Challenge Dataset Statistics

Group	N_{Playlist}	H_{avg}	N_{Track}	N_{Artist}
K=0	1000	29	0	0
K=1	1000	23	932	715
K=5	2000	55	6790	2762
K=10	2000	53	11877	4096
K=25	2000	126	22507	6253
K=100	2000	88	53552	11517

the playlist). The aim of the challenge was to generate a list of 500 *recommended tracks* for each of these playlists, based on the partial information available. Additionally, each playlist contained a number of *holdout tracks* that were known only to the organizers of the challenge. The submissions of the participants were evaluated and ranked based on the correspondence between the recommended tracks and the holdout tracks [1].

The playlists in the challenge dataset can be divided into ten distinct challenge categories based on the type of the provided partial information [1], namely:

- G1: Playlists with a title only
- G2: Playlists with a title and the first track
- G3: Playlists with a title and the first five tracks
- G4: Playlists with first five tracks (no title)
- G5: Playlists with a title and the first ten tracks
- G6: Playlists with first ten tracks (no title)
- G7: Playlists with a title and the first 25 tracks
- G8: Playlists with a title and 25 random tracks
- G9: Playlists with a title and the first 100 tracks
- G10: Playlists with a title and 100 random tracks

Table 1 shows the distribution of playlists, the average number of holdout tracks (H_{avg}), the number of unique seed tracks (N_{Track}), and the number of unique artists (N_{Artist}) present the challenge dataset, grouped according to the number of seed tracks K .

3 FRAMEWORK

In this section, we describe the framework for automatic playlist continuation (APC). The framework consists of three steps: first a collection of multiple different collaborative filtering models are extracted in the *collaborative filtering stage*, then, the predictions of the collaborative filtering models are combined into a single relevance prediction per playlist-track combination in the *composition stage*, and finally, the recommendations are generated in the *playlist continuation stage*. We now continue with describing these stages in detail.

Collaborative Filtering Stage

The task of collaborative filtering is to predict the utility of items (tracks) to a particular *context* (playlist) based on vector similarities between these entities extracted from data [6]. This *context* can be based on different aspects of a playlist. For example, in item-item collaborative filtering, the context is based on the tracks that are already present in the playlist. A total of four collaborative models were built in order to capture different contexts:

track-track model (M_u) models the relevance of a given track for a given playlist based on the set of tracks that are currently present in the playlist. This model is a traditional item-item collaborative filtering model.

word-track model (M_w) models the relevance of a given track for a given playlist based on the name of the playlist. This collaborative filter that models the relation between words in the playlist name and the occurrence of tracks when a playlist contains this word in the playlist title. The words are extracted from the playlist names by splitting the playlist name on the space character (i.e. ' '), transforming the results to lowercase, and removing punctuation marks.

album-track model (M_{al}) models the relevance of a given track for a given playlist based on the albums from the tracks that are currently present in the playlist. This collaborative filter models the relation between the set of albums of the tracks that are currently in the playlist and the occurrence of tracks when these albums are in the playlist.

artist-track model (M_{ar}) models the relevance of a given track for a given playlist based on the artists the created the tracks that are currently present in the playlist. This collaborative filter models the relation between the set of albums of the tracks that are currently in the playlist and the occurrence of tracks when these albums are in the playlist.

Composition Stage

The output of every collaborative filter is combined in a final ranking model (M_c) using a weighted sum given by:

$$M_c = W_u * M_u + W_w * M_w + W_{al} * M_{al} + W_{ar} * M_{ar} \quad (1)$$

where W_u , W_w , W_{al} and W_{ar} are real-valued weights in range $[0, 1]$.

The best configuration of weights is found using an optimization procedure, such as Tree-structured Parzen Estimator (TPE) [4]. We experiment with two types of weighting schemes: 1) global weights (optimized over all instances) and

2) local weights (optimized separately for each challenge category). We describe the procedure to determine the weights W_u , W_w , W_{al} , and W_{ar} in detail in Section 4.

Playlist Continuation Stage

To determine the recommended tracks for a given playlist, we filter the tracks on $M_c > 0$ and then sort the tracks in descending order based on their M_c value, using the M_c value that uses the weights that we found in the composition stage. However, it can be the case that fewer than 500 tracks have a value of M_c that is larger than zero, in which case the requirement of recommending 500 songs would not be satisfied. To improve the order of tracks in the recommendations ranking and to guarantee a total 500 recommended tracks for every playlist, we apply two post-processing steps.

The first post-processing step aims at completing the albums that are currently already present in the playlists. This is motivated by the fact that a reasonable number of playlists in the dataset contained exactly all the tracks of a single album, and we found the M_c to be insufficient to properly detect this scenario and complete the album for playlists that contain a high number of tracks from the same album. When the ratio of the number of tracks from the number of distinct albums that are currently in the playlist exceeds a threshold m (where m is a tunable parameter), we first recommend all the tracks from that remaining album before recommending the tracks based on M_c .

As a second post-processing step, to fulfill the requirement of recommending exactly 500 tracks, we append the list of recommended tracks with the most popular tracks in the dataset in decreasing order of overall frequency until the list of recommended tracks contains exactly 500 tracks.

4 MODEL SELECTION

We evaluate the model instantiations using a combination of three measures R-precision, NDCG, and CLICKS, which are the same three measures that are used by the RecSys challenge organizers to score the submissions. We select the best performing model instantiation for submission.

In this section we present the evaluation measures, the procedure for optimizing the models' parameters, the procedure and the results for selecting the best model. The framework was implemented in Python and can be found at [9] under open source license.

Evaluation measures

The R-precision, defined as:

$$\text{R-precision} = \frac{|G \cap R|}{|G|} \quad (2)$$

where G is the set of ground truth (holdout) tracks, and R is the set of recommended tracks. The notation $|\cdot|$ denotes the number of elements in the set.

The Normalized discounted cumulative gain (NDCG), defined as:

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}} \quad (3)$$

where:

$$\text{DCG} = \text{rel}_1 + \sum_{i=2}^{|R|} \frac{\text{rel}_i}{\log_2(i+1)} \quad (4)$$

$$\text{IDCG} = 1 + \sum_{i=2}^{|G|} \frac{1}{\log_2(i+1)} \quad (5)$$

The Recommended Songs CLICKS metric, that mimics a Spotify feature for track recommendation where ten tracks are presented at a certain time to the user as the suggestion to complete the playlist. This metric captures the number of refreshes needed before a relevant track is encountered, and is defined as:

$$\text{CLICKS} = \frac{\text{argmin}_i \{R_i : R_i \in G\} - 1}{10} \quad (6)$$

While NDCG and CLICKS were calculated based on the track-level agreement between the holdout tracks and the recommended tracks, R-precision was calculated on the artist-level agreement. In other words, it was considered sufficient if the artist of a recommended track matched the artist of a holdout track.

Approaches

We tested three instantiations of the proposed framework, namely:

- composition via global weights;
- composition via local weights, without album completion (i.e., $m = \infty$);
- composition via local weights, where the album completion threshold m is optimized through the same procedure as optimizing the weights.

As a baseline, we compared the results to a simple popularity-based model, where the recommendation list is created based on the overall popularity of songs in a non-personalized manner.

Model Optimization

For each of the tested approaches, the weights for combining the collaborative filters needed to be optimized. Furthermore, in the variant with album completion, the song to album ratio m was optimized. To this end, we extracted an optimization dataset (D_{opt}) containing 10k playlists (playlists